



ALPHASIGNAL × TIGER DATA · WORKSHOP

Agent observability on Postgres

Monitoring AI agents when there are hundreds of them

THE SETUP

One agent, you read the logs. Hundreds of agent workflows across an org is a different system.



This is not a logging problem

EVERY AGENT RUN EMITS

prompts · tool calls · model responses · latency · tokens · cost
errors · retries · outcomes

Most teams put that in a log pipeline.

It's time-series and events.

Timestamped, append-only, aggregated over windows, queried by trend.



Where this sits

You'll probably buy a trace tool. Build this once anyway, so you know what it's doing and what it can't do for you.

LIVE DEBUGGING

**Langfuse, Braintrust,
Logfire, Phoenix**

one trace, right now

30-day retention

DURABLE AGGREGATE LAYER

What we build today

every trace, over a year

as long as you want



THE QUESTIONS YOU WILL BE ASKED

Three months from now

THE BILL

It tripled

Nobody can say which agent did it.

THE DRIFT

The p99 has been climbing

Nobody knows when it started.

THE AUDIT

Finance asks about last quarter

The traces were deleted on day 30.



DO NOT INVENT COLUMN NAMES

OpenTelemetry GenAI semantic conventions

<code>gen_ai.operation.name</code>	chat execute_tool invoke_agent plan
<code>gen_ai.agent.name</code>	which agent
<code>gen_ai.conversation.id</code>	session / thread
<code>gen_ai.usage.input_tokens</code>	prompt tokens
<code>gen_ai.usage.output_tokens</code>	completion tokens
<code>gen_ai.usage.cache_read.input_tokens</code>	cache hits
<code>gen_ai.tool.name</code>	which tool
<code>error.type</code>	why it failed

TWO CAVEATS

These are still Development status, and OTel defines no cost attribute — that one is borrowed from Arize's OpenInference.



The shape everyone converges on

ONE FLAT TABLE

session

trace · one agent run

invoke_agent

← root, carries rolled-up cost

chat

← an LLM call

execute_tool

← a tool call

chat

THAT IS THE WHOLE TRICK

**One flat table.
parent_span_id.**

Langfuse, LangSmith, Phoenix, Weave, Braintrust — all the same.

Nobody uses closure tables. Nobody runs recursive CTEs on the hot path.



2,000,000

agent spans · 300 agents · 12 tenants · 30 days

Generated server-side. Nothing to download.



WHAT WE ARE ACTUALLY ASKING THE DATA

Four questions

Nothing exotic. Every team asks these.



01 LATENCY

Which agents are slow? p99 by agent.



02 COST

What is each customer costing us?



03 RELIABILITY

Which tools fail?



04 VOLUME

How many distinct conversations per day?



TWO TOOLS, NOT ONE

Two different fixes

HYPERTABLE

Partitions by time

Helps queries that filter by time.

Chunks the planner can exclude before it reads anything.

COLUMNSTORE

Column-by-column, compressed

Helps queries that touch few columns across many rows.

segmentby: agent_name, operation — low cardinality, and what we filter on.

They are not the same thing and they do not help the same queries.



MEASURED, NOT ASSUMED

Two out of four

	plain	columnstore	
storage	329 MB	105 MB	3.1x smaller
tool error rates	733 ms	148 ms	4.9x faster
p99 by agent	1,554 ms	651 ms	2.4x faster
cost by tenant	981 ms	867 ms	barely moved
distinct conversations	2,482 ms	3,040 ms	WORSE
last 24h, time-filtered	full scan	4.5 ms	hypertable, not columnstore



THE BEST IDEA IN THIS SESSION

You cannot average a p99

Store a sketch, not a number. Sketches merge.

WHAT GOES WRONG

bucket 1 p99 = 900 ms

bucket 2 p99 = 400 ms

$\text{avg}(900, 400) = 650 \text{ ms}$

the real p99 = 880 ms

WHAT TO DO INSTEAD

```
percentile_agg(duration_ms)
-- in the rollup
```

```
approx_percentile(0.99,
    rollup(latency))
-- reading it back
```

Same trick for distinct counts: hyperloglog instead of a hash set.



A MISTAKE WORTH TELLING

Rollups are shaped like questions

FIRST ATTEMPT

hyperloglog in the 5-minute rollup, grouped by
agent × tenant × operation

512 ms

239,060 sketches to merge
from a 44 MB rollup

SECOND ATTEMPT

one rollup, grouped only by day

1.2 ms

31 sketches
632 kB

400× faster, 70× smaller, same number.



Three techniques, three numbers

	plain	columnstore	rollup
p99 by agent	1,554 ms	651 ms	215 ms
cost by tenant	981 ms	867 ms	119 ms
distinct conversations	2,482 ms	3,040 ms	1.2 ms

HOW WRONG IS "APPROX"?

p99 sketch 9,584 vs 9,579 exact

conversations 20,000 vs 20,001 exact



DEMO

Live

Every panel but one reads a continuous aggregate.

The retry-storm table scans raw spans on purpose — that is the one question worth paying a full scan for.



RETENTION

Throw the raw data away

Nobody debugs a specific agent run from six weeks ago.

Everybody asks what agents cost per customer last quarter.

RAW SPANS

2,013,057 → 494,988 rows

102 MB → 26 MB

keep 7 days

DAILY ROLLUP

178,052 rows

back to day 1

keep for years



What to take away

- | Agent telemetry is time-series, not logs. Model it that way.
- | Match the OTel GenAI conventions so you are not inventing a schema.
- | Storage tricks do not help every query. Measure, do not assume.
- | Sketches are what make pre-aggregation honest.
- | Rollups are shaped like questions. Build several; they are cheap.
- | Keep the aggregates long after you drop the raw spans.

github.com/timescale/agent-observability-workshop



Go run it

`github.com/timescale/`
`agent-observability-workshop`

WHAT IT COSTS TO TRY

0.5 CPU / 2 GB — the smallest paid size. Well inside new-account trial credit.

THE FREE TIER WORKS TOO

It runs there, but shared CPU roughly **doubles every timing** you saw today.

